Note

nolva

2021 年 6 月 4 日

目录

1 torch训练加速方法 **2**
1.1 *学习率策略 2
1.2 Dataloader 2
1.3 batch_size 2
1.4 自动混合精度 2
1.5 cudNN benchmarking 3
1.6 CPU and GPU 3
1.7 checkpoint 3
1.8 梯度累加 3
1.9 多GPU分布式数据并行 3
1.10 梯度设为None而不是0 3
1.11 减少数据拷贝,即使销毁计算图 3
1.12 调试工具 4
1.13 梯度裁剪 4
1.14 BatchNorm前不使用bias 4
1.15 对输入归一化 4
1.16 使用 JIT融合point-wise的操作 4

A Snipptes **5**
A.1 随机数 5
A.2 PCA 5
A.3 Metrics 5
A.3.1 分类 5

1 torch训练加速方法

1.1 *学习率策略

这个可能要依据文献做
比如torch.optim.lr_scheduler.CyclicLR和torch.optim.lr_scheduler.OneCycleLR

1.2 Dataloader

pin_memory=True,保证内存驻留，不交换到磁盘中

1.3 batch_size

经验法则，学习率随batch大小提升；大batch可能导致泛化能力比小batch差

1.4 自动混合精度

pytorch1.6起；与在所有地方都使用单精度(FP32)相比，某些操作可以在半精度(FP16)下运行得更快，而且不会损失精度

```
# enable 16-bit on the model and the optimizer
model, optimizers = amp.initialize(model, optimizers, opt_level='O2')

# when doing .backward, let amp do it so it can scale the loss
with amp.scale_loss(loss, optimizer) as scaled_loss:
   scaled_loss.backward()
```

```
import torch
# Creates once at the beginning of training
scaler = torch.cuda.amp.GradScaler()

for data, label in data_iter:
   optimizer.zero_grad()
   # Casts operations to mixed precision
   with torch.cuda.amp.autocast():
      loss = model(data)

   # Scales the loss, and calls backward()
   # to create scaled gradients
   scaler.scale(loss).backward()

   # Unscales gradients and calls
   # or skips optimizer.step()
   scaler.step(optimizer)
```

```
    # Updates the scale for next iteration
    scaler.update()
```

1.5 cudNN benchmarking

```
torch.backends.cudnn.benchmark = True
```

1.6 CPU and GPU

小心使用.cpu(),.cuda(),.item(),.numpy()，用detach替代

用device=直接分配到device上

1.7 checkpoint

```
torch.utils.checkpoint
```

1.8 梯度累加

多次backward后，再进行step

1.9 多GPU分布式数据并行

```
torch.nn.DistributedDataParallel
# 而不是 torch.nn.DataParallel
# 不同部分放在不同的gpu上
self.encoder.cuda(0)
self.decoder.cuda(1)
```

1.10 梯度设为None而不是0

使用.zero_grad(set_to_none=True)而不是.zero_grad()。这样做会让内存分配器去处理梯度，而不是主动将它们设置为0。正如在文档中所说的那样，这会导致产生一个适度的加速，所以不要期待任何奇迹。

注意，这样做并不是没有副作用的！关于这一点的详细信息请查看文档。

1.11 减少数据拷贝,即使销毁计算图

torch.tensor() 会拷贝数据，如果你有一个numpy数组，你想转为tensor，使用 torch.as_tensor() 或是 torch.from_numpy() 来避免拷贝数据。

使用losses.append(loss.item())

1.12 调试工具

1.13 梯度裁剪

1.14 BatchNorm前不使用bias

1.15 对输入归一化

1.16 使用 JIT融合point-wise的操作

如果你有point-wise的操作，你可以使用PyTorch JIT将它们合并成一个FusionGroup，这样就可以在单个核上启动，而不是像默认情况下那样在多个核上启动。你还可以节省一些内存的读写。

Szymon Migacz展示了如何使用@torch.jit脚本装饰器来融合GELU中的操作，例如：

```
# 速度提高了5倍
@torch.jit.script
def fused_gelu(x):
    return x * 0.5 * (1.0 + torch.erf(x / 1.41421))
```

A Snipptes

A.1 随机数

```
def set_random_seed(mySeed=0):
    torch.manual_seed(mySeed) # CPU
    torch.cuda.manual_seed(mySeed) #当前GPU
    torch.cuda.manual_seed_all(mySeed) #所有GPU
    torch.backends.cudnn.deterministic = True #使cuDNN操作具有确定性
    torch.backends.cudnn.benchmark = False
    np.random.seed(mySeed)
    random.seed(mySeed)
```

A.2 PCA

```
def applyPCA(X, numComponents):
    newX = np.reshape(X, (-1, X.shape[2]))
    pca = PCA(n_components=numComponents, whiten=True)#whiten使每个特征具有相同的方差
    newX = pca.fit_transform(newX)
    newX = np.reshape(newX, (X.shape[0], X.shape[1], numComponents))
    return newX
```

A.3 Metrics

A.3.1 分类

```
from sklearn.metrics import  classification_report,accuracy_score,confusion_matrix
# 需要把每个batch的pred,target连接起来
classification = classification_report(target, pred, digits=4)
print(classification)

def AA_andEachClassAccuracy(confusion_matrix):
    counter = confusion_matrix.shape[0]
    list_diag = np.diag(confusion_matrix)
    list_raw_sum = np.sum(confusion_matrix, axis=1)
    each_acc = np.nan_to_num(truediv(list_diag, list_raw_sum))
    average_acc = np.mean(each_acc)
    return each_acc, average_acc

def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)
```

```python
    _, pred = output.topk(maxk, 1, True, True)  # 返回最大的k个结果（按最大到小排序）
    pred = pred.t()  # 转置
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0, keepdim=True)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res

class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count
```